

DOCUMENTO TECNICO DO PRODUTO

LexDoc

Referencia

Tecnica

Completa

Plataforma SaaS de Gestao Documental Juridica
Especializada no Direito Mocambicano

Documento tecnico exhaustivo contendo todas as especificacoes de arquitectura, base de dados, API endpoints, componentes frontend, logica de negocio, sistema de IA e configuracao necessarias para replicar o produto LexDoc na sua totalidade.

Indice

Placeholder for table of contents

0

1. Visao Geral do Produto

1.1 Identificacao do Produto

Nome: LexDoc

Versao: 0.2.0

Licenca: Privada

Tipo: Plataforma SaaS de Gestao Documental Juridica para Mocambique

1.2 Publico-Alvo

O LexDoc foi concebido para atender as necessidades de:

- Escritorios de advocacia em Mocambique
- Consultorias juridicas
- Tribunais e orgaos de administracao da justica
- Departamentos juridicos de empresas

1.3 Idioma Principal

O idioma principal da plataforma e o Portugues juridico mocambicano, com todo o interface, mensagens de erro, notificacoes e documentacao disponiveis neste idioma.

1.4 Funcionalidades Principais

O LexDoc oferece as seguintes funcionalidades nucleares:

Gestao de Processos: Criacao, edicao, acompanhamento e encerramento de processos juridicos com atribuicao de advogados, prazos e documentos associados.

Gestao de Clientes: Cadastro completo de clientes com dados pessoais, contacto e vinculo a processos juridicos.

Gestao de Documentos: Upload, versionamento e organizacao de documentos por processo com metadados e historico.

Prazos Processuais: Controlo de prazos com suporte a extracao automatica via IA, notificacoes e vista de calendario.

Centro de IA (LexAssistent): Assistente juridico com chat inteligente, geracao de documentos e extracao de prazos.

Base de Conhecimento Juridico: Artigos juridicos com sistema RAG para pesquisa contextualizada na legislacao mocambicana.

Auditoria: Registro imutavel de todas as acoes do sistema com mascara de dados pessoais.

Relatorios: Paineis analiticos com estatisticas de processos, clientes, documentos e prazos.

Gestao de Utilizadores (RBAC): Sistema de controlo de acesso baseado em funcoes com quatro niveis hierarquicos.

Convites: Sistema de convites por token para adicionar novos membros a firma.

Templates de Processo: Modelos pre-definidos para criacao rapida de processos com checklists.

Quadro Kanban: Vista visual de processos em colunas por estado com suporte a drag-and-drop.

Calendario: Vista mensal de prazos e eventos com navegacao entre meses.

Tarefas: Sistema de notas e tarefas com filtros por entidade e prioridade.

Notificacoes: Centro de notificacoes em tempo real para alertas e lembretes.

Pesquisa Global: Barra de pesquisa unificada para processos, clientes e documentos.

1.5 Visao Geral da Plataforma

O LexDoc e uma aplicacao single-page (SPA) construida sobre o framework Next.js com App Router. A navegacao interna e gerida por um store Zustand (useNavStore) que renderiza diferentes vistas dentro de um unico componente de pagina (page.tsx). O dashboard principal possui 16 abas de navegacao que cobrem todas as funcionalidades da plataforma. A API RESTful esta disponivel em /api/v1/ com autenticao JWT e multi-tenancy por firm_id.

2. Stack Tecnologico

A tabela seguinte apresenta todas as tecnologias utilizadas no LexDoc, incluindo versoes e propositos especificos.

Tecnologia	Versao / Detalhe
Framework Web	Next.js 16.1.1 (App Router)
Linguagem	TypeScript 5
Estilizacao	Tailwind CSS 4 + shadcn/ui (New York style)
Base de Dados	SQLite via Prisma ORM 6.11.1
Estado do Cliente	Zustand 5.0.6
Estado do Servidor	TanStack Query 5.82.0
Animacoes	Framer Motion 12.23.2
Graficos	Recharts 2.15.4
Markdown	react-markdown 10.1.0 + remark-gfm 4.0.1
Drag and Drop	@dnd-kit/core 6.3.1

Tecnologia	Versao / Detalhe
Formularios	react-hook-form 7.60.0 + @hookform/resolvers 5.1.1
Validacao	Zod 4.0.2
Datas	date-fns 4.1.0 + date-fns-tz 3.2.0
Autenticacao (servidor)	jsonwebtoken 9.0.3 + bcryptjs 3.0.3
Icones	lucide-react 0.525.0
Tema (dark/light)	next-themes 0.4.6
Notificacoes Toast	sonner 2.0.6
SDK de IA	z-ai-web-dev-sdk 0.0.17
Internacionalizacao	next-intl 4.3.4
Fontes	Geist (Sans + Mono) via next/font/google

3. Arquitectura do Sistema

3.1 Arquitectura Geral

O LexDoc e uma aplicacao single-page (SPA) com client-side routing implementado via Zustand (useNavStore). Todas as vistas sao renderizadas dentro do componente `src/app/page.tsx`, que verifica o estado de autenticacao e decide qual vista apresentar: `LoginView`, `RegisterView` ou `DashboardView`. A navegacao interna do dashboard nao utiliza o sistema de file routing do Next.js; em vez disso, e gerida por abas no componente `DashboardView`.

3.2 API RESTful

A API esta organizada sob o caminho `/api/v1/` com autenticacao JWT. Todas as rotas protegidas utilizam o middleware `authenticateRequest` que extrai o token Bearer do header `Authorization`. O sistema implementa multi-tenancy por `firm_id`, onde todas as queries incluem um filtro pela firma do utilizador autenticado.

3.3 Fluxo de Navegacao

O componente `page.tsx` implementa o seguinte fluxo:

1. Verifica se o utilizador esta autenticado (`auth.store`)
2. Se nao autenticado: renderiza `LoginView`, `RegisterView` ou `ForgotPassword`
3. Se autenticado: renderiza `DashboardView`
4. `DashboardView` gere 16 abas internas via estado `local`

3.4 Abas do Dashboard

O DashboardView possui 16 abas de navegacao, cada uma renderizando um componente especifico:

Aba	Componente	Descricao
painel	DashboardHome	Painel principal com widgets, estatisticas e feed de atividade
ia	AIHubView	Centro de IA com 4 sub-abas: chat, gerar, extrair prazos, analisar
tarefas	TaskManager	Gestao de notas e tarefas com filtros por entidade
processos	ProcessesView	Lista e gestao completa de processos juridicos
modelos	TemplatesView	Templates de processo com checklists
quadro	KanbanBoard	Quadro Kanban visual com drag-and-drop
documentos	DocumentsView	Gestao de documentos com upload e versionamento
clientes	ClientsView	Cadastro e gestao de clientes
base-conheciment o	KnowledgeView	Base de conhecimento juridico com RAG
prazos	DeadlinesView	Gestao de prazos processuais
calendario	CalendarView	Calendario mensal de prazos
utilizadores	UsersView	Gestao de utilizadores com RBAC
convites	InvitationsView	Sistema de convites por token
relatorios	ReportsView	Paineis analiticos e graficos
auditoria	AuditView	Registo de auditoria imutavel
notificacoes	NotificationsCenter	Centro de notificacoes

3.5 Navegacao na Sidebar

A sidebar apresenta itens de navegacao filtrados por role do utilizador (RBAC). Os itens sao animados com Framer Motion e possuem um indicador visual de aba ativa. A sidebar pode ser colapsada (w-64 para w-16) e em dispositivos moveis e apresentada como overlay.

4. Esquema da Base de Dados

O LexDoc utiliza SQLite como base de dados, gerida pelo Prisma ORM. O esquema consiste em 17 modelos com relacoes bem definidas. Todos os modelos possuem campos created_at e updated_at com valores padrao now(). O multi-tenancy e implementado atraves do campo firm_id presente na maioria dos modelos.

4.1 Firm (firms)

Descricao: Raiz do multi-tenancy. Cada firma representa um escritorio de advocacia.

Tabela: @@map("firms")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
name	String	NOT NULL	-
slug	String	UNIQUE	-
logo_url	String?	nullable	-
address	String?	nullable	-
phone	String?	nullable	-
email	String?	nullable	-
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Relacoes: User[] (um-para-muitos), Client[] (um-para-muitos), etc.

4.2 User (users)

Descricao: Utilizadores do sistema com papeis RBAC: ADMIN, ADVOGADO, SECRETARIO, CLIENT.

Tabela: @@map("users")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
email	String	NOT NULL, UNIQUE	-
password_hash	String	NOT NULL	-
name	String	NOT NULL	-
role	Enum(Role)	NOT NULL	CLIENT
avatar_url	String?	nullable	-
phone	String?	nullable	-
bi_number	String?	nullable	-
nif	String?	nullable	-
is_active	Boolean	NOT NULL	true
login_attempts	Int	NOT NULL	0
locked_until	DateTime?	nullable	-
created_at	DateTime	NOT NULL	now()

Campo	Tipo	Restricoes	Default
updated_at	DateTime	NOT NULL	now()

Enum Role: ADMIN, ADVOGADO, SECRETARIO, CLIENT

Relacoes: Firm (muitos-para-um), RefreshToken[], AuditLog[], ProcessAssignment[], Note[], Invitation[], AIConversation[]

Indexes: UNIQUE(email), INDEX(firm_id)

4.3 RefreshToken (refresh_tokens)

Descricao: Armazenamento de refresh tokens com hash SHA-256 para seguranca.

Tabela: @@map("refresh_tokens")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
user_id	String	NOT NULL, FK -> users.id	-
token_hash	String	NOT NULL, UNIQUE	-
expires_at	DateTime	NOT NULL	-
created_at	DateTime	NOT NULL	now()

Relacoes: User (muitos-para-um). Indexes: UNIQUE(token_hash), INDEX(user_id).

4.4 AuditLog (audit_logs)

Descricao: Registro de auditoria imutavel (sem update/delete) para todas as acoes do sistema.

Tabela: @@map("audit_logs")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
user_id	String?	nullable	-
action	String	NOT NULL	-
entity_type	String?	nullable	-
entity_id	String?	nullable	-
details	Json?	nullable	-
ip_address	String?	nullable	-
user_agent	String?	nullable	-
created_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um), User (muitos-para-um, opcional). Indexes: INDEX(firm_id), INDEX(created_at), INDEX(entity_type, entity_id).

4.5 Client (clients)

Descricao: Clientes da firma com dados pessoais completos.

Tabela: @@map("clients")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
name	String	NOT NULL	-
email	String?	nullable	-
phone	String?	nullable	-
address	String?	nullable	-
bi_number	String?	nullable	-
nif	String?	nullable	-
type	String?	nullable	-
notes	String?	nullable	-
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um), LegalProcess[]. Indexes: INDEX(firm_id).

4.6 LegalProcess (legal_processes)

Descricao: Processos juridicos com estado, tipo e vinculacao a clientes.

Tabela: @@map("legal_processes")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
client_id	String	NOT NULL, FK -> clients.id	-
number	String	NOT NULL	-
title	String	NOT NULL	-
description	String?	nullable	-
type	String?	nullable	-
status	Enum(ProcessStatus)	NOT NULL	ATIVO
priority	Enum(Priority)	NOT NULL	MEDIA
court	String?	nullable	-

Campo	Tipo	Restricoes	Default
judge	String?	nullable	-
opposing_party	String?	nullable	-
value	Float?	nullable	-
started_at	DateTime?	nullable	-
closed_at	DateTime?	nullable	-
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Enum ProcessStatus: ATIVO, SUSPENSO, ENCERRADO, ARQUIVADO

Enum Priority: BAIXA, MEDIA, ALTA, URGENTE

Relacoes: Firm (muitos-para-um), Client (muitos-para-um), ProcessAssignment[], Document[], Deadline[], ProcessNote[]

Indexes: INDEX(firm_id), INDEX(client_id), INDEX(status), @@unique([firm_id, number])

4.7 ProcessAssignment (process_assignments)

Descricao: Atribuicao de advogados a processos juridicos.

Tabela: @@map("process_assignments")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
process_id	String	NOT NULL, FK -> legal_processes.id	-
user_id	String	NOT NULL, FK -> users.id	-
role	String	NOT NULL	ADVOGADO
assigned_at	DateTime	NOT NULL	now()

Relacoes: LegalProcess (muitos-para-um), User (muitos-para-um). Indexes: @@unique([process_id, user_id]).

4.8 Document (documents)

Descricao: Gestao de documentos com versionamento e vinculacao a processos.

Tabela: @@map("documents")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
process_id	String	NOT NULL, FK -> legal_processes.id	-
uploaded_by	String	NOT NULL, FK -> users.id	-

Campo	Tipo	Restricoes	Default
title	String	NOT NULL	-
description	String?	nullable	-
file_name	String	NOT NULL	-
file_size	Int	NOT NULL	0
file_type	String	NOT NULL	-
file_path	String	NOT NULL	-
version	Int	NOT NULL	1
category	String?	nullable	-
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um), LegalProcess (muitos-para-um), User (muitos-para-um). Indexes: INDEX(firm_id), INDEX(process_id).

4.9 Deadline (deadlines)

Descricao: Prazos processuais com suporte a criacao manual e extracao automatica via IA.

Tabela: @@map("deadlines")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
process_id	String	NOT NULL, FK -> legal_processes.id	-
title	String	NOT NULL	-
description	String?	nullable	-
due_date	DateTime	NOT NULL	-
status	Enum(DeadlineStatus)	NOT NULL	PENDENTE
priority	Enum(Priority)	NOT NULL	MEDIA
source	String	NOT NULL	MANUAL
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Enum DeadlineStatus: PENDENTE, CONCLUIDO, ATRASADO, CANCELADO

Enum Priority: BAIXA, MEDIA, ALTA, URGENTE

Relacoes: Firm (muitos-para-um), LegalProcess (muitos-para-um). Indexes: INDEX(firm_id), INDEX(process_id), INDEX(due_date), INDEX(status).

4.10 Note (notes)

Descricao: Sistema de notas e tarefas com filtros por tipo de entidade e prioridade.

Tabela: @@map("notes")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
user_id	String	NOT NULL, FK -> users.id	-
title	String	NOT NULL	-
content	String?	nullable	-
entity_type	String?	nullable	-
entity_id	String?	nullable	-
priority	Enum(Priority)	NOT NULL	MEDIA
is_completed	Boolean	NOT NULL	false
completed_at	DateTime?	nullable	-
due_date	DateTime?	nullable	-
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um), User (muitos-para-um). Indexes: INDEX(firm_id), INDEX(user_id), INDEX(entity_type, entity_id).

4.11 KnowledgeArticle (knowledge_articles)

Descricao: Base de conhecimento juridico para o sistema RAG de pesquisa.

Tabela: @@map("knowledge_articles")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
title	String	NOT NULL	-
content	String	NOT NULL	-
source	String?	nullable	-
category	String?	nullable	-
tags	String?	nullable (JSON array)	-
is_active	Boolean	NOT NULL	true
created_at	DateTime	NOT NULL	now()

Campo	Tipo	Restricoes	Default
updated_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um). Indexes: INDEX(firm_id), INDEX(category).

4.12 ProcessTemplate (process_templates)

Descricao: Templates de processo com checklists pre-definidos.

Tabela: @@map("process_templates")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
name	String	NOT NULL	-
description	String?	nullable	-
type	String?	nullable	-
checklist	Json?	nullable (array)	-
is_active	Boolean	NOT NULL	true
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um). Indexes: INDEX(firm_id).

4.13 ProcessNote (process_notes)

Descricao: Notas especificas de processos juridicos.

Tabela: @@map("process_notes")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
process_id	String	NOT NULL, FK -> legal_processes.id	-
user_id	String	NOT NULL, FK -> users.id	-
content	String	NOT NULL	-
is_private	Boolean	NOT NULL	false
created_at	DateTime	NOT NULL	now()

Relacoes: LegalProcess (muitos-para-um), User (muitos-para-um). Indexes: INDEX(process_id).

4.14 Invitation (invitations)

Descricao: Sistema de convites por token para adicionar membros a firma.

Tabela: @@map("invitations")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
invited_by	String	NOT NULL, FK -> users.id	-
email	String	NOT NULL	-
role	Enum(Role)	NOT NULL	-
token	String	NOT NULL, UNIQUE	-
expires_at	DateTime	NOT NULL	-
accepted_at	DateTime?	nullable	-
status	Enum(InvitationStatus)	NOT NULL	PENDING
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Enum InvitationStatus: PENDING, ACCEPTED, EXPIRED, CANCELLED

Relacoes: Firm (muitos-para-um), User (invited_by, muitos-para-um). Indexes: UNIQUE(token), INDEX(firm_id), INDEX(email).

4.15 AIConversation (ai_conversations)

Descricao: Conversas de chat com a IA LexAssistent.

Tabela: @@map("ai_conversations")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
user_id	String	NOT NULL, FK -> users.id	-
title	String	NOT NULL	-
created_at	DateTime	NOT NULL	now()
updated_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um), User (muitos-para-um), AIMessage[]. Indexes: INDEX(firm_id), INDEX(user_id).

4.16 AIMessage (ai_messages)

Descricao: Mensagens individuais dentro de conversas de IA.

Tabela: @@map("ai_messages")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
conversation_id	String	NOT NULL, FK -> ai_conversations.id	-
role	String	NOT NULL	-
content	String	NOT NULL	-
sources	Json?	nullable	-
created_at	DateTime	NOT NULL	now()

Relacoes: AIConversation (muitos-para-um). Indexes: INDEX(conversation_id).

4.17 AIGeneration (ai_generations)

Descricao: Registo de geracoes de documentos por IA.

Tabela: @@map("ai_generations")

Campo	Tipo	Restricoes	Default
id	String	PRIMARY KEY, id	cuid()
firm_id	String	NOT NULL, FK -> firms.id	-
user_id	String	NOT NULL, FK -> users.id	-
type	String	NOT NULL	-
title	String	NOT NULL	-
content	String	NOT NULL	-
prompt	String?	nullable	-
process_id	String?	nullable, FK -> legal_processes.id	-
created_at	DateTime	NOT NULL	now()

Relacoes: Firm (muitos-para-um), User (muitos-para-um), LegalProcess (muitos-para-um, opcional). Indexes: INDEX(firm_id), INDEX(user_id).

5. API Endpoints

O LexDoc expoe uma API RESTful completa sob o caminho /api/v1/. A tabela seguinte lista todos os 48+ endpoints disponiveis, organizados por modulo.

5.1 Autenticacao (Auth)

Metodo	Endpoint	Descricao
POST	/api/v1/auth/login	Autenticacao de utilizador
POST	/api/v1/auth/register	Registo de novo utilizador
POST	/api/v1/auth/refresh	Renovacao de token de acesso
POST	/api/v1/auth/logout	Terminar sessao
POST	/api/v1/auth/forgot-password	Pedido de recuperacao de senha
POST	/api/v1/auth/reset-password	Redefinicao de senha

5.2 Utilizadores (Users)

Metodo	Endpoint	Descricao
GET	/api/v1/users	Listar utilizadores da firma
POST	/api/v1/users	Criar novo utilizador
GET	/api/v1/users/[id]	Obter detalhes do utilizador
PATCH	/api/v1/users/[id]	Atualizar utilizador
PATCH	/api/v1/users/[id]/deactivate	Desativar utilizador

5.3 Clientes (Clients)

Metodo	Endpoint	Descricao
GET	/api/v1/clients	Listar clientes
POST	/api/v1/clients	Criar cliente
GET	/api/v1/clients/[id]	Obter detalhes do cliente
PATCH	/api/v1/clients/[id]	Atualizar cliente

5.4 Processos (Processes)

Metodo	Endpoint	Descricao
GET	/api/v1/processes	Listar processos
POST	/api/v1/processes	Criar processo
GET	/api/v1/processes/[id]	Obter detalhes do processo
PATCH	/api/v1/processes/[id]	Atualizar processo
PATCH	/api/v1/processes/[id]/close	Encerrar processo
PATCH	/api/v1/processes/[id]/status	Alterar estado do processo
GET	/api/v1/processes/[id]/timeline	Obter timeline do processo

Metodo	Endpoint	Descricao
GET	/api/v1/processes/[id]/deadlines	Listar prazos do processo
POST	/api/v1/processes/[id]/deadlines	Criar prazo no processo
GET	/api/v1/processes/[id]/notes	Listar notas do processo
POST	/api/v1/processes/[id]/notes	Criar nota no processo

5.5 Documentos (Documents)

Metodo	Endpoint	Descricao
GET	/api/v1/documents	Listar documentos
POST	/api/v1/documents	Fazer upload de documento
GET	/api/v1/documents/[id]	Obter detalhes do documento
PATCH	/api/v1/documents/[id]	Atualizar documento
DELETE	/api/v1/documents/[id]	Eliminar documento

5.6 Prazos (Deadlines)

Metodo	Endpoint	Descricao
GET	/api/v1/deadlines	Listar prazos
POST	/api/v1/deadlines	Criar prazo
GET	/api/v1/deadlines/[id]	Obter detalhes do prazo
PATCH	/api/v1/deadlines/[id]	Atualizar prazo
GET	/api/v1/deadlines/calendar	Obter prazos para calendario

5.7 Notas (Notes)

Metodo	Endpoint	Descricao
GET	/api/v1/notes	Listar notas
POST	/api/v1/notes	Criar nota
PATCH	/api/v1/notes/[id]	Atualizar nota
DELETE	/api/v1/notes/[id]	Eliminar nota

5.8 Base de Conhecimento (Knowledge)

Metodo	Endpoint	Descricao
GET	/api/v1/knowledge	Listar artigos
POST	/api/v1/knowledge	Criar artigo
GET	/api/v1/knowledge/[id]	Obter artigo
PATCH	/api/v1/knowledge/[id]	Atualizar artigo
DELETE	/api/v1/knowledge/[id]	Eliminar artigo
GET	/api/v1/knowledge/stats	Estatisticas da base de conhecimento

5.9 Templates

Metodo	Endpoint	Descricao
GET	/api/v1/templates	Listar templates
POST	/api/v1/templates	Criar template
GET	/api/v1/templates/[id]	Obter template
PATCH	/api/v1/templates/[id]	Atualizar template
DELETE	/api/v1/templates/[id]	Eliminar template
POST	/api/v1/templates/[id]/use	Usar template para criar processo

5.10 Inteligencia Artificial (AI)

Metodo	Endpoint	Descricao
POST	/api/v1/ai/chat	Enviar mensagem ao chat da IA
GET	/api/v1/ai/chat	Listar conversas de IA
GET	/api/v1/ai/conversations/[id]	Obter conversa especifica
DELETE	/api/v1/ai/conversations/[id]	Eliminar conversa
POST	/api/v1/ai/generate	Gerar documento com IA
GET	/api/v1/ai/generate/list	Listar geracoes
GET	/api/v1/ai/generate/[id]	Obter geracao especifica
DELETE	/api/v1/ai/generate/[id]	Eliminar geracao
POST	/api/v1/ai/extract-deadlines	Extrair prazos de texto com IA
POST	/api/v1/ai/analyze	Analisar documento com IA

5.11 Firma (Firm)

Metodo	Endpoint	Descricao
GET	/api/v1/firm/settings	Obter configuracoes da firma
PATCH	/api/v1/firm/settings	Atualizar configuracoes da firma
GET	/api/v1/firm/members	Listar membros da firma

5.12 Convites (Invitations)

Metodo	Endpoint	Descricao
GET	/api/v1/invitations	Listar convites
POST	/api/v1/invitations	Criar convite
GET	/api/v1/invitations/[token]	Obter detalhes do convite
DELETE	/api/v1/invitations/[token]	Cancelar convite
POST	/api/v1/invitations/[token]/accept	Aceitar convite

5.13 Outros Endpoints

Metodo	Endpoint	Descricao
GET	/api/v1/audit/logs	Consultar logs de auditoria
GET	/api/v1/reports/overview	Relatorio geral
GET	/api/v1/search	Pesquisa global
GET	/api/v1/stats/dashboard	Estatisticas do dashboard
GET	/api/v1/activity/recent	Atividade recente
GET	/api/v1/notifications	Listar notificacoes
GET	/api/v1/profile	Perfil do utilizador
PATCH	/api/v1/profile	Atualizar perfil
PATCH	/api/v1/profile/password	Alterar senha
GET	/api/v1/export/clients	Exportar clientes
GET	/api/v1/export/processes	Exportar processos
GET	/api/v1/export/audit	Exportar auditoria
GET	/api/v1/export/report-pdf	Exportar relatorio PDF
POST	/api/v1/import/clients	Importar clientes

6. Modulos de Biblioteca (src/lib/)

A camada de logica de negocio e utilidades esta organizada em modulos TypeScript sob src/lib/. Cada modulo tem uma responsabilidade especifica e bem definida.

6.1 auth.ts

Autenticacao JWT e hashing de senhas

Responsavel por toda a logica de autenticacao do sistema. Implementa:

JWT Access Tokens: Validade de 15 minutos, assinados com JWT_SECRET

JWT Refresh Tokens: Validade de 7 dias, assinados com JWT_REFRESH_SECRET

Bcrypt: Hashing de senhas com 10 rounds de salt

SHA-256: Hashing de refresh tokens antes do armazenamento

6.2 api-auth.ts

Middleware de autenticacao de requests

Middleware authenticateRequest que extrai o token Bearer do header Authorization, verifica a validade do JWT e injeta o payload decodificado no request. Retorna 401 se o token for invalido ou ausente.

6.3 rbac.ts

Controlo de acesso baseado em funcoes

Implementa a hierarquia de papeis e funcoes de verificacao de acesso:

Hierarquia: ADMIN(4) > ADVOGADO(3) > SECRETARIO(2) > CLIENT(1)

hasRole(userRole, requiredRole): Verifica se o utilizador possui o role minimo

canAccessResource(userRole, resourceOwnerRole): Verifica acesso a recursos de outros utilizadores

canManageRole(managerRole, targetRole): Verifica se um gestor pode gerir um determinado role

6.4 audit.ts

Registo de auditoria fire-and-forget

Sistema de auditoria assincrono que regista acoes sem bloquear a resposta ao cliente. Caracteristicas:

Fire-and-forget: Registo assincrono via Prisma, sem await na rota principal

Mascara de PII: Campos sensiveis sao removidos dos detalhes antes do registo

Campos mascarados: bi_number, nif, phone, address, password_hash, mfa_secret

6.5 rate-limit.ts

Limitador de taxa in-memory

Implementacao de rate limiting baseada em Map em memoria:

Estrutura: Map com chave IP e array de timestamps

Limpeza: Automatica a cada 60 segundos

Uso principal: Endpoints de login (5 req/60s por IP)

6.6 rag-search.ts

Motor de pesquisa RAG

Motor de pesquisa semantica para a base de conhecimento juridico:

Stop words: Lista customizada de palavras de paragem em portugues

Scoring V2.0: Fiabilidade da fonte com ponderacao por tipo de legislacao

Pontuacao ponderada: Titulo (5pts), Tags (4pts), Fonte (3pts), Conteudo (1pt)

Normalizacao: Tokens normalizados para comparacao case-insensitive

6.7 lexassist-prompt.ts

Prompt system do LexAssistent v2.0

Contem o prompt de sistema e templates de resposta da IA. Inclui:

Governanca V2.0: Hierarquia de 4 niveis para garantia de qualidade

Protocolo Mata-Ilusion: Verificacao pre-conclusao para sancoes criminais

Zero Confusao Lusofona: Deteccao e correcao automatica de termos PT/BR

Formato IRAC: Questao, Norma, Aplicacao, Conclusao, Riscos, Fontes

Templates de documentos: 8 tipos: peticao-inicial, contestacao, contrato-trabalho, procuracao, parecer-juridico, notificacao, requerimento, recurso

6.8 pagination.ts

Utilitarios de paginacao

Funcoes auxiliares para paginacao padronizada:

parsePagination(query): Extrai page e limit dos query params

buildPaginationMeta(total, page, limit): Gera metadados de paginacao

calcSkip(page, limit): Calcula offset para query Prisma

6.9 notes-db.ts

Operacoes CRUD para notas

Modulo de acesso a dados para o sistema de notas. Suporta filtragem por entity_type e entity_id, alem de operacoes completas de criar, listar, atualizar e eliminar notas.

6.10 reset-token-store.ts

Armazenamento de tokens de reset

Armazenamento in-memory de tokens de recuperacao de senha:

Estrutura: Map com token como chave e dados de expiracao

Expiracao: Tokens expiram apos 5 minutos

Limpeza: Automatica de tokens expirados

6.11 db.ts

Singleton Prisma

Instancia singleton do PrismaClient com cache global para ambiente de desenvolvimento. Evita multiplas instancias em modo dev com hot-reloading do Next.js.

6.12 utils.ts

Utilitario cn()

Funcao cn() que combina clsx (para condicionais de classes) com tailwind-merge (para fusao inteligente de classes Tailwind), garantindo que nao ha conflitos de classes CSS.

6.13 api-client.ts

Cliente API tipado

Cliente HTTP completo e tipado para comunicacao com a API:

Interceptors: Interceptacao automatica de respostas 401

Refresh queue: Fila de refresh token para pedidos simultaneos

Tipagem: Interfaces TypeScript para todos os payloads

Base URL: Configuravel via variavel de ambiente

7. Componentes Frontend (src/components/)

Os componentes do frontend estão organizados em quatro categorias principais: Views (vistas principais), Auth (autenticação), Dashboard (componentes do painel) e Shared/UI (componentes reutilizáveis).

7.1 Views (3 componentes)

Componentes de nível superior que representam as vistas principais da aplicação:

Componente	Descrição
LoginView.tsx	Vista de autenticação com formulário de login e link para recuperação de senha
RegisterView.tsx	Vista de registro de novo utilizador com indicador de força de senha
DashboardView.tsx	Vista principal do dashboard com 16 abas, sidebar e header

7.2 Auth (5 componentes)

Componentes relacionados com autenticação e gestão de conta:

Componente	Descrição
LoginForm.tsx	Formulário de login com validação e suporte a bloqueio de conta
RegisterForm.tsx	Formulário de registro com validação completa
ForgotPasswordForm.tsx	Formulário de pedido de recuperação de senha
ResetPasswordForm.tsx	Formulário de redefinição de senha com token
PasswordStrengthIndicator.tsx	Indicador visual da força da senha

7.3 Dashboard (31 componentes)

Componentes do painel principal, incluindo todas as vistas de gestão:

Componente	Descrição
DashboardHome.tsx	Painel principal com widgets, estatísticas e feed de atividade
AIHubView.tsx	Centro de IA com 4 sub-abas (~1452 linhas de código)
AIChatPanel.tsx	Painel de chat IA flutuante acessível de qualquer vista
ProcessesView.tsx	Lista e gestão de processos com filtros e busca
ClientsView.tsx	Gestão de clientes com importação/exportação
UsersView.tsx	Gestão de utilizadores com RBAC
DeadlinesView.tsx	Gestão de prazos com filtros por estado
DocumentsView.tsx	Gestão de documentos com upload e versionamento
CalendarView.tsx	Calendário mensal de prazos com navegação

Componente	Descricao
KnowledgeView.tsx	Base de conhecimento juridico com RAG
TemplatesView.tsx	Templates de processo com checklists
KanbanBoard.tsx	Quadro Kanban com drag-and-drop via @dnd-kit
TaskManager.tsx	Gestao de notas e tarefas com filtros
ReportsView.tsx	Paineis analiticos com graficos Recharts
AuditView.tsx	Vista de logs de auditoria com filtros
InvitationsView.tsx	Gestao de convites por token
NotificationsCenter.tsx	Centro de notificacoes completo
SearchBar.tsx	Barra de pesquisa global unificada
NotificationBell.tsx	Sino de notificacoes com badge
ProfileDialog.tsx	Dialogo de perfil do utilizador
FirmSettingsDialog.tsx	Dialogo de configuracoes da firma
InvitationDialog.tsx	Dialogo de criacao de convites
KnowledgeArticleDialog.tsx	Dialogo de criacao/edicao de artigos
OnboardingGuide.tsx	Guia de introducao para novos utilizadores
KeyboardShortcutsDialog.tsx	Dialogo de atalhos de teclado
WidgetSettings.tsx	Configuracao de widgets do dashboard
QuickActionsFAB.tsx	Botao flutuante de acoes rapidas
ActivityFeed.tsx	Feed de atividade recente
DashboardSkeleton.tsx	Skeleton loading para o dashboard
ProcessTimeline.tsx	Timeline de eventos do processo
ProcessNotesPanel.tsx	Painel de notas do processo
NotesPanel.tsx	Painel de notas geral
AcceptInvitationView.tsx	Vista de aceitacao de convite

7.4 Shared (1 componente)

Componente	Descricao
MarkdownRenderer.tsx	Renderizador de Markdown com suporte a GFM (tabelas, listas de tarefas, etc.)

7.5 UI - Biblioteca shadcn/ui (40+ componentes)

O LexDoc utiliza a biblioteca shadcn/ui com estilo New York, que inclui os seguintes componentes reutilizaveis:

accordion, alert, alert-dialog, avatar
badge, breadcrumb, button, calendar
card, carousel, chart, checkbox
collapsible, command, context-menu, data-table
dialog, drawer, dropdown-menu, form
hover-card, input, input-otp, label
menubar, navigation-menu, pagination, popover
progress, radio-group, resizable, scroll-area
select, separator, sheet, sidebar
skeleton, slider, switch, table
tabs, textarea, toast, toaster
sonner, toggle, toggle-group, tooltip
aspect-ratio

8. Sistema de IA - LexAssistent v2.0

O LexAssistent é o sistema de inteligência artificial do LexDoc, concebido especificamente para o contexto jurídico moçambicano. Implementa múltiplas camadas de segurança e governança para garantir respostas precisas e confiáveis.

8.1 Governança V2.0

O sistema de governança implementa uma hierarquia de 4 níveis para garantia de qualidade das respostas:

- **Nível 1 - RAG MZ:** Prioridade absoluta para fontes moçambicanas (Constituição, CODJUR, Código Civil, Código Penal, leis sectoriais).
- **Nível 2 - Filtro Geográfico:** Rejeição automática de leis e jurisprudência de outros países lusófonos (Brasil, Portugal, Angola, etc.).
- **Nível 3 - Cronologia:** Verificação temporal da legislação, priorizando versões mais recentes e revogações.
- **Nível 4 - Silêncio Seguro:** Quando não há confiança suficiente na resposta, o sistema admite a limitação em vez de inventar informação.

8.2 Protocolo Mata-Ilusão

Protocolo de segurança para questões criminais que exige verificação pré-conclusão antes de mencionar quaisquer sanções penais. O sistema deve verificar a existência de circunstâncias atenuantes, agravantes,

e a tipologia exata do crime antes de apresentar quaisquer penas possíveis. Isto previne conclusões precipitadas que poderiam ter consequências graves para clientes.

8.3 Zero Confusão Lusófona

Sistema de detecção e correção automática que identifica e corrige termos específicos do Português do Brasil (PT-BR) que poderiam ser confundidos com o Português de Moçambique (PT-MZ). Exemplos incluem a substituição de termos como 'estuprador' por 'criminoso sexual', 'delegado' por 'magistrado do Ministério Público', e correções de terminologia processual.

8.4 Motor RAG

O motor de Retrieval-Augmented Generation implementa:

- Tokenização com lista customizada de 200+ stop words em português
- Scoring com ponderação: Título (5 pts), Tags (4 pts), Fonte (3 pts), Conteúdo (1 pt)
- Fiabilidade da fonte com bonus para legislação oficial moçambicana
- Normalização case-insensitive de tokens
- Retorno de top 5 artigos mais relevantes com pontuação

8.5 Formato de Resposta IRAC

Todas as respostas do LexAssistent seguem o formato IRAC adaptado para o contexto jurídico moçambicano:

Secção	Conteúdo
Questão	Identificação clara da questão jurídica
Norma	Citação da legislação aplicável (com referência ao diploma legal)
Aplicação	Aplicação da norma ao caso concreto
Conclusão	Conclusão fundamentada com recomendações
Riscos	Identificação de riscos e alternativas
Fontes	Lista de fontes consultadas (legislação, jurisprudência, doutrina)
Nota de Auditoria	Meta-dados sobre o nível de confiança e fontes utilizadas

8.6 Geração de Documentos

O LexAssistent suporta a geração de 8 tipos de documentos jurídicos:

Tipo (slug)	Nome
peticao-inicial	Petição Inicial

Tipo (slug)	Nome
contestacao	Contestacao
contrato-trabalho	Contrato de Trabalho
procuracao	Procuracao
parecer-juridico	Parecer Juridico
notificacao	Notificacao
requerimento	Requerimento
recurso	Recurso

8.7 Extracao de Prazos

Funcionalidade de extracao automatica de prazos processuais a partir de textos juridicos. Utiliza o LLM para identificar datas, prazos e eventos temporais, com opcao de criacao automatica na base de dados.

8.8 Limites de Taxa

Rate limiting especifico para endpoints de IA, por utilizador:

Endpoint	Limite
Chat	20 pedidos/hora por utilizador
Geracao de documentos	10 pedidos/hora por utilizador
Extracao de prazos	15 pedidos/hora por utilizador

8.9 Validacao Pos-LLM

Apos cada resposta do LLM, o sistema executa validacoes adicionais:

- Deteccao de contaminacao por termos PT-BR (Portugues do Brasil)
- Injecao automatica de disclaimer juridico
- Verificacao de conformidade com o formato IRAC
- Validacao de citacoes legais contra fontes conhecidas

9. Sistema de Autenticacao e Seguranca

9.1 Tokens JWT

O sistema utiliza um esquema de autenticacao com dois tipos de token:

- **Access Token:** JWT com validade de 15 minutos, utilizado para autenticação de requests. Assinado com JWT_SECRET.
- **Refresh Token:** JWT com validade de 7 dias, utilizado para renovação do access token. Assinado com JWT_REFRESH_SECRET. Armazenado como hash SHA-256 na base de dados.

9.2 Hashing de Senhas

As senhas são hashadas utilizando bcrypt com 10 rounds de salt. O bcryptjs é utilizado para compatibilidade com o ambiente Node.js/Bun.

9.3 Bloqueio de Conta

Sistema de proteção contra ataques de força bruta:

- Após 5 tentativas falhadas consecutivas, a conta é bloqueada por 15 minutos
- O campo login_attempts registra o número de tentativas
- O campo locked_until registra o momento em que o bloqueio expira
- Após o bloqueio expirar, as tentativas são resetadas no primeiro login bem-sucedido

9.4 Rate Limiting

Proteção contra abuso implementada em múltiplas camadas:

- **Login:** Limite de 5 pedidos por 60 segundos por endereço IP
- **Endpoints de IA:** Limites por utilizador (20 chat/hr, 10 generate/hr, 15 extract/hr)
- **Implementação:** Baseada em Map em memória com limpeza automática

9.5 Mascara de PII em Auditoria

Para proteção de dados pessoais nos logs de auditoria, os seguintes campos são automaticamente removidos dos detalhes antes do registo:

Campo	Descrição
bi_number	Numero do Bilhete de Identidade
nif	Numero de Identificação Fiscal
phone	Numero de telefone
address	Morada completa
password_hash	Hash da senha
mfa_secret	Segredo de autenticação multi-fator

9.6 Refresh Tokens Seguros

Os refresh tokens são armazenados na base de dados como hashes SHA-256, nunca em texto claro. O token original é retornado apenas ao cliente no momento da autenticação.

9.7 Tokens de Reset

Tokens de recuperação de senha são armazenados em memória (não persistidos em base de dados) com expiração de 5 minutos e limpeza automática de tokens expirados.

9.8 Sistema de Convites

Sistema de convites baseado em tokens únicos:

- O ADMIN cria convites com email e role pretendido
- Um token único é gerado para cada convite
- O convidado acede /invite/[token] para aceitar o convite
- Após aceitação, uma conta é criada na firma do convidante
- Convites expiram após um período definido (configurável)

9.9 RBAC - Controlo de Acesso Baseado em Funções

O sistema implementa 4 níveis hierárquicos de acesso:

Role	Nível	Permissões
ADMIN	4	Acesso total: gestão de utilizadores, configurações, todos os módulos
ADVOGADO	3	Gestão de processos atribuídos, documentos, prazos, IA
SECRETARIO	2	Gestão de clientes, documentos, agendamento
CLIENT	1	Acesso apenas de leitura aos seus próprios processos e documentos

10. Estado do Cliente (Stores)

O estado do lado do cliente é gerido por dois stores Zustand, localizados em src/stores/.

10.1 auth.store.ts

Store principal de autenticação com persistência em localStorage:

Propriedade	Tipo	Descrição
accessToken	string null	Token de acesso JWT
refreshToken	string null	Token de refresh JWT (persistido)

Propriedade	Tipo	Descricao
user	User null	Dados do utilizador autenticado
isAuthenticated	boolean	Estado de autenticacao derivado
isLoading	boolean	Estado de carregamento
restoreSession()	async function	Restaura sessao do localStorage
setAuth()	function	Define tokens e dados do utilizador
logout()	function	Limpa estado e localStorage
updateUser()	function	Atualiza dados do utilizador

Persistencia: accessToken e refreshToken sao persistidos em localStorage via Zustand persist middleware.

10.2 nav.store.ts

Store de navegacao que controla a vista atual da aplicacao:

Propriedade	Tipo	Descricao
currentView	string	Vista atual: login register dashboard forgot-password
navigate(view)	function	Altera a vista atual

11. Layout e Navegacao

11.1 DashboardView - Layout Principal

O DashboardView implementa o layout principal da aplicacao com a seguinte estrutura:

```
min-h-screen flex
+-- Sidebar (fixa, w-64/w-16 colapsavel, cor #0f0f1e)
+-- Area Principal (flex-1, overflow-y-auto)
+-- Header (backdrop-blur, fixo no topo)
+-- Conteudo da aba (com padding)
+-- Footer (fixo no fundo)
```

11.2 Sidebar

A sidebar de navegacao possui as seguintes caracteristicas:

- Posicao fixa com largura w-64 expandida e w-16 colapsada
- Cor de fundo escura: #0f0f1e
- Itens de navegacao filtrados por role do utilizador (RBAC)
- Indicador visual de aba ativa animado com Framer Motion

- Em dispositivos moveis: apresentada como overlay com fundo semi-transparente
- Transicao suave de expansao/colapso

11.3 Header

O header do dashboard inclui:

- Efeito backdrop-blur para transparencia com desfoque
- Barra de pesquisa global
- Sino de notificacoes com badge de contagem
- Alternancia de tema (dark/light)
- Acesso a atalhos de teclado

11.4 Footer

O footer do dashboard e sticky e inclui:

- Relogio com fuso horario Africa/Maputo (CAT, UTC+2)
- Badge de versao (v0.2.0)
- Indicador de estado online

11.5 Design Responsivo

A aplicacao implementa design responsivo com:

- Sidebar como overlay em dispositivos moveis
- Pesquisa compacta em ecras menores
- Padding responsivo para o conteudo principal
- Layout adaptavel para tablet e desktop

11.6 Transicoes de Abas

A troca entre abas utiliza AnimatePresence do Framer Motion com motion.div para transicoes suaves de entrada e saida.

11.7 Cadeia de Altura Flex para o AI Hub

Para garantir que o AI Hub (centro de IA) ocupe toda a altura disponivel, e implementada uma cadeia de altura flex:

```
main (h-screen) > motion.div (h-full) > AIHubView > tab content > AssistantChatTab
```

12. Configuracao e Deploy

12.1 Ambiente de Desenvolvimento

Para iniciar o ambiente de desenvolvimento:

```
bun run dev # Inicia o servidor de desenvolvimento na porta 3000
# Output redirecionado para dev.log
```

12.2 Build de Producao

O processo de build utiliza o Next.js com output standalone:

```
next build # Gera build com output standalone
# Ficheiros estaticos copiados para o directorio de output
```

12.3 Base de Dados

O LexDoc utiliza SQLite como base de dados com Prisma ORM:

```
bun run db:push # Sincroniza o schema Prisma com a base de dados
```

O ficheiro de configuracao do Prisma esta em prisma/schema.prisma e define todos os 17 modelos, relacoes e indexes descritos na Secao 4.

12.4 Variaveis de Ambiente

As seguintes variaveis de ambiente sao necessarias:

Variavel	Exemplo	Descricao
TZ	Africa/Maputo	Fuso horario para todas as operacoes de data
JWT_SECRET	(chave secreta)	Chave para assinatura de access tokens
JWT_REFRESH_SECRET	(chave secreta)	Chave para assinatura de refresh tokens
DATABASE_URL	file:./custom.db	Caminho para a base de dados SQLite

12.5 Gateway - Caddy

O deploy utiliza o Caddy como reverse proxy com suporte a multi-porta:

- Configuracao definida no ficheiro Caddyfile na raiz do projeto
- Utiliza o parametro XTransformPort para roteamento de multiplas portas
- HTTPS automatico via Let's Encrypt

12.6 Fuso Horário

Todos os endpoints da API definem explicitamente TZ=Africa/Maputo (CAT, UTC+2) para garantir consistencia nas operacoes de data e hora. As bibliotecas date-fns e date-fns-tz sao utilizadas para formatacao e conversao de fusos horarios no frontend.